

Object Relation Model (ORM)

 by Praveen Kumar

What is ORM (Object Relational Mapping)?

ORM (Object Relational Mapping) is a programming technique that allows developers to **map Java objects to relational database tables** and perform database operations **without writing SQL directly**.

In Simple Words

👉 ORM lets you **work with Java objects**, while it **automatically handles SQL, tables, rows, joins, and CRUD operations** behind the scenes.

Problems Without ORM (Using JDBC)

Problem	Explanation
Too much SQL	Manual queries everywhere
Tight coupling	Java code depends heavily on DB schema
Boilerplate code	Connection, PreparedStatement, ResultSet
Hard maintenance	Any DB change → code change
No OOP benefits	Tables ≠ Objects

Benefits of ORM

Benefit	Description
Object-oriented	Work with Java classes
Less SQL	Mostly auto-generated
Database independent	Switch DB with minimal changes
Productivity	Faster development
Caching	Improves performance
Transaction management	Easy commit / rollback

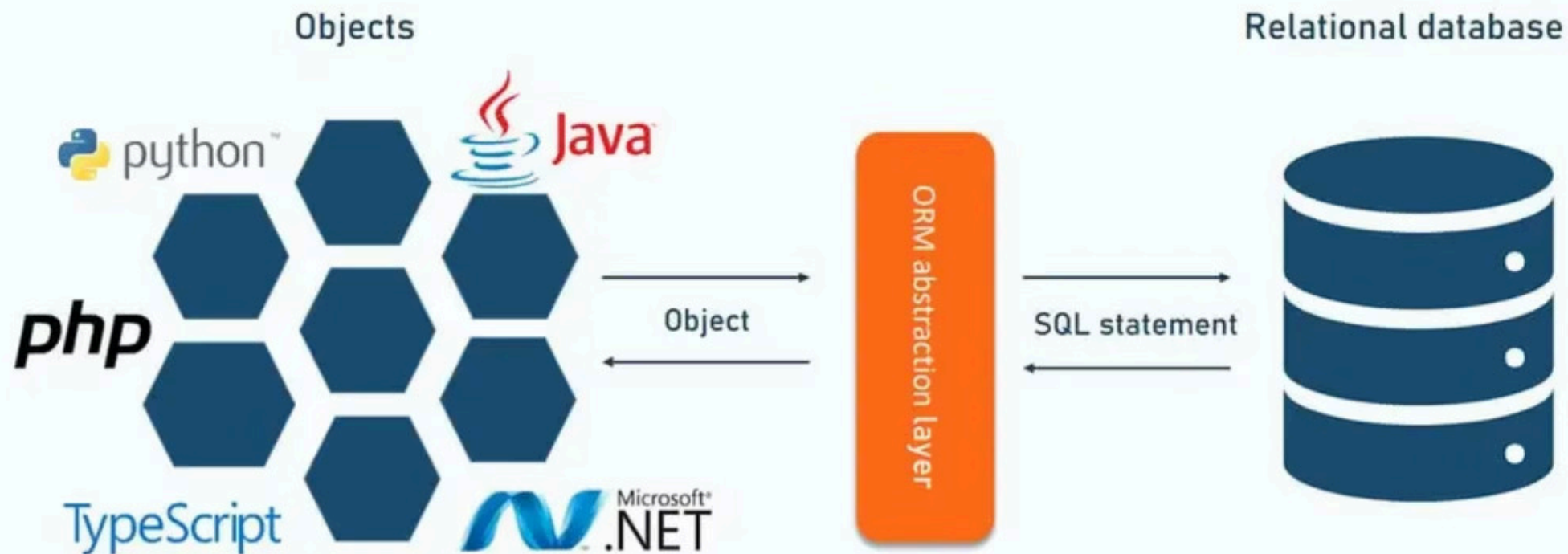
Core ORM Concept

Java World	Database World
Class	Table
Object	Row
Field	Column
Primary Key	@Id
Relationship	@OneToMany, @ManyToOne

Popular ORM Frameworks

Language	ORM Framework
Java	Hibernate
Java	JPA
.NET	Entity Framework
Python	Django ORM, SQLAlchemy

OBJECT-RELATIONAL MAPPING PROCESS



 altexsoft

What is JPA?

JPA (Java Persistence API) is a **Java specification** that defines **how Java objects are mapped to relational database tables** and **how data persistence operations are performed**.

👉 JPA **does NOT** provide implementation

👉 JPA defines **rules, interfaces, and annotations**

Simple Definition

JPA is a **standard ORM specification** for managing relational data in Java applications using **objects instead of SQL**.

What is Hibernate?

Hibernate is a **Java-based ORM (Object Relational Mapping) framework** that allows developers to **map Java objects to relational database tables** and perform **CRUD operations without writing SQL manually**.

👉 Hibernate automatically:

- Converts **Java objects** → **SQL**
- Converts **DB rows** → **Java objects**
- Manages **transactions, caching, relationships, and queries**

Hibernate is a framework that lets you work with the database using Java objects instead of SQL queries.

Hibernate vs ORM vs JPA (Very Important)

Term	Meaning
ORM	Concept (Object ↔ Table mapping)
Hibernate	ORM implementation
JPA	ORM specification

 Hibernate **implements JPA**, but also provides **extra powerful features** beyond JPA.

@Query in Spring Data JPA?

@Query allows you to **write custom queries manually** instead of relying only on:

- Derived query methods (findByNameAndDept)
- Default CRUD methods

It is used when:

- Query is **complex**
- You need **joins**
- You need **aggregations**
- You want **better control**
- Derived method names become unreadable

- @Query("QUERY_STRING")

```
returnType methodName(parameters);
```

Types of Queries Supported by @Query

Type	Description
JPQL	Object-oriented, entity-based
Native SQL	Database-specific SQL
Named Parameters	:name syntax
Positional Parameters	?1, ?2
Update/Delete Queries	DML operations
Projection Queries	Partial data
Pagination & Sorting	Pageable support

JPQL vs Native Query (Very Important)

JPQL	Native SQL
Uses Entity name	Uses Table name
Uses field names	Uses column names
DB independent	DB dependent
Object-oriented	SQL-oriented

Basic JPQL Query using @Query

Example: Find employee by name

```
@Query("SELECT e FROM Employee e WHERE e.name = :name")  
Employee findByName(@Param("name") String name);
```

Using Positional Parameters

```
@Query("SELECT e FROM Employee e WHERE e.department = ?1")  
List<Employee> findByDepartment(String dept);
```

Multiple Conditions in @Query

```
@Query("SELECT e FROM Employee e WHERE e.department = :dept AND e.active = true")  
List<Employee> findActiveEmployees(@Param("dept") String dept);
```

LIKE Queries (Search Functionality)

```
@Query("SELECT e FROM Employee e WHERE e.name LIKE %:keyword%")  
List<Employee> searchByName(@Param("keyword") String keyword);
```

Sorting in @Query

```
@Query("SELECT e FROM Employee e ORDER BY e.salary DESC")  
List<Employee> findAllOrderBySalaryDesc();
```

Aggregation Queries

MAX salary

```
@Query("SELECT MAX(e.salary) FROM Employee e")  
Double findMaxSalary();
```

COUNT employees by department

```
@Query("SELECT COUNT(e) FROM Employee e WHERE e.department = :dept")  
Long countByDepartment(@Param("dept") String dept);
```


Projection Queries (Partial Data)

Returning Selected Columns

```
@Query("SELECT e.name, e.salary FROM Employee e")  
List<Object[]> findNameAndSalary();
```

Accessing Data

```
for(Object[] obj : list) {  
    String name = (String) obj[0];  
    Double salary = (Double) obj[1];  
}
```

Interface-based Projection (Recommended)

```
public interface EmployeeView {  
    String getName();  
    double getSalary();  
}
```

```
@Query("SELECT e.name AS name, e.salary AS salary FROM Employee e")  
List<EmployeeView> fetchEmployeeView();
```

- ✓ Clean
- ✓ Type-safe

Native Queries using @Query

Example: Native SQL

```
@Query(value = "SELECT * FROM employees WHERE department = :dept", nativeQuery = true)  
List<Employee> findByDeptNative(@Param("dept") String dept);
```

 Use when:

- DB-specific functions
- Stored procedures
- Complex joins

Update Queries using @Query

⚠ Very important concept

Step 1: Use @Modifying

```
@Modifying
@Query("UPDATE Employee e SET e.salary = :salary WHERE e.id = :id")
int updateSalary(@Param("salary") double salary,
                 @Param("id") Long id);
```

Step 2: Use Transaction

```
@Transactional
public void updateEmpSalary(Long id, double salary) {
    repository.updateSalary(salary, id);
}
```

📌 Without @Modifying → query fails

📌 Without @Transactional → update not committed

Dynamic Queries using @Query

```
@Query("""  
SELECT e FROM Employee e  
WHERE (:dept IS NULL OR e.department = :dept)  
AND (:active IS NULL OR e.active = :active)  
""")  
List<Employee> search(@Param("dept") String dept,  
                     @Param("active") Boolean active);
```

- ✓ Optional filters
- ✓ Cleaner than building queries manually

“Exception Handling in REST Controllers” Mean?

In a **REST application**, exception handling means:

Converting **Java exceptions** into **meaningful HTTP responses** (status code + JSON body) that clients can understand.

REST controllers **do not return views (HTML)**.

They return **data (JSON/XML)** → so exceptions must also return **data**, not error pages.

How Exceptions Occur in REST Controllers

Exceptions can occur at **any layer**:

Layer	Example
Controller	Invalid request body / path variable
Validation	@Valid fails
Service	Business rule violation
Repository	Data not found
Framework	Type mismatch, HTTP errors

All these exceptions finally **reach the REST controller layer**.

What Happens If You Do NOTHING? (Default Behavior ❌)

If no exception handling is configured:

- Spring Boot returns:
 - HTTP **500**
 - Long stack trace
 - Unclear error message

Example response:

```
{
  "timestamp": "...",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/employees/10"
}
```

- ❌ Not user-friendly
- ❌ Not secure
- ❌ Not REST-professional

REST Exception Handling Flow (Very Important)

```
graph TD; Client1[Client Postman / Frontend] --> VC[@RestController]; VC --> SE[Service throws Exception]; SE --> EPU[Exception propagates upward]; EPU --> RCA[@RestControllerAdvice]; RCA --> EH[@ExceptionHandler method]; EH --> RE[ResponseEntity status + JSON]; RE --> Client2[Client];
```

Client (Postman / Frontend)

|

v

@RestController

|

Service throws Exception

|

Exception propagates upward

|

@RestControllerAdvice

|

@ExceptionHandler method

|

ResponseEntity (status + JSON)

|

Client

Core Principles of REST Exception Handling

- ✓ Use **HTTP status codes**
- ✓ Return **JSON error response**
- ✓ Centralize handling
- ✓ Never expose stack traces
- ✓ Separate success & error responses

```
@RestController  
@RequestMapping("/employees")  
public class EmployeeController {  
    @GetMapping("/{id}")  
    public Employee getEmployee(@PathVariable int id) {  
        if (id <= 0) {  
            throw new IllegalArgumentException("Invalid employee id");  
        }  
        return service.getEmployee(id);  
    }  
}
```

This throws an exception, but does NOT handle it properly yet.

Custom Exception (Best Practice)

Why custom exceptions?

- Clear meaning
- Better mapping to HTTP status
- Easy to maintain

```
public class ResourceNotFoundException extends RuntimeException {  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

Throw Exception from Service Layer (Correct Design)

```
public Employee getEmployee(int id) {  
    return repository.findById(id)  
        .orElseThrow(() ->  
            new ResourceNotFoundException(  
                "Employee not found with id: " + id));  
}
```

📌 Controllers stay **clean**

📌 Business logic stays in **service**

Global Exception Handling using `@RestControllerAdvice`

This is the **MOST IMPORTANT** concept.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

`@RestControllerAdvice` =

- `@ControllerAdvice` + `@ResponseBody`
- Applies to **ALL REST controllers**

Handling a Specific Exception

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(
    ResourceNotFoundException ex) {

    ErrorResponse error = new ErrorResponse(
        404,
        ex.getMessage(),
        LocalDateTime.now()
    );

    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
}
```

Standard Error Response Model (Recommended)

```
public class ErrorResponse {  
  
    private int status;  
    private String message;  
    private LocalDateTime timestamp;  
  
    // constructors, getters, setters  
}
```

Sample JSON Output

```
{  
  "status": 404,  
  "message": "Employee not found with id: 5",  
  "timestamp": "2026-01-21T10:45:30"  
}
```


What is Data Validation in Spring Data JPA?

Data Validation ensures that:

- Only **correct, meaningful, and safe data** is persisted to the database.
- Invalid input is **caught early** (before DB insert/update).
- Business rules and constraints are **enforced consistently**.

Spring Data JPA uses **Bean Validation (JSR-380 / Jakarta Validation)** integrated with:

- Entity classes
- Spring MVC
- Spring Boot auto-configuration

Validation Layers in a Spring Application

Validation can happen at **multiple levels**:

Layer	Purpose
Client-side	UI validation (HTML5, JS, Angular, React)
Controller layer	Request validation (@Valid)
Service layer	Business rule validation
Entity layer (JPA)	Persistent constraint validation
Database layer	DB constraints (NOT NULL, UNIQUE, FK)

👉 **Spring Data JPA focuses mainly on Entity + Controller validation**

Bean Validation (JSR-380 / Jakarta Validation)

Spring uses **Jakarta Validation** (earlier javax.validation).

Core Concepts

- Constraints are declared using **annotations**
- Validation is triggered automatically
- Errors are collected as **ConstraintViolation**

Common Validation Annotations

4.1 Basic Constraints

Annotation	Description
@NotNull	Value must not be null
@NotEmpty	Not null + not empty (collections, strings)
@NotBlank	Not null + trimmed length > 0
@Size(min, max)	Length or size constraint
@Min, @Max	Numeric limits
@Positive, @Negative	Sign constraints
@Email	Valid email format
@Pattern	Regex-based validation

Example Entity with Validation

```
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long empId;

    @NotBlank(message = "Employee name is mandatory")
    @Size(min = 3, max = 30, message = "Name must be 3-30 characters")
    private String name;

    @NotNull(message = "Salary cannot be null")
    @Min(value = 10000, message = "Salary must be at least 10000")
    private Double salary;

    @Email(message = "Invalid email format")
    @Column(unique = true)
    private String email;

    @NotBlank
    private String department;
}
```

✓ These constraints are checked **before insert/update**

How Validation Works in Spring Data JPA (Flow)

Step-by-Step Flow

1. Client sends data (JSON / form)
2. Controller receives request
3. `@Valid` triggers validation
4. Hibernate Validator checks annotations
5. If validation fails:
 - Exception is thrown
6. If validation passes:
 - JPA persists entity

Enabling Validation in Controller Layer

6.1 Using @Valid

```
@PostMapping("/employees")
public ResponseEntity<Employee> saveEmployee(
    @Valid @RequestBody Employee employee) {

    return ResponseEntity.ok(employeeRepository.save(employee));
}
```

What Happens?

- @Valid → triggers validation
- If any constraint fails → MethodArgumentNotValidException

Handling Validation Errors Gracefully

Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationErrors(
        MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();

        ex.getBindingResult().getFieldErrors()
            .forEach(error ->
                errors.put(error.getField(), error.getDefaultMessage())
            );

        return ResponseEntity.badRequest().body(errors);
    }
}
```

Sample Error Response

```
{
  "name": "Employee name is mandatory",
  "salary": "Salary must be at least 10000"
}
```


Validation at JPA Entity Level (Automatic)

Spring Data JPA integrates with **Hibernate Validator**.

Whenever you call:

```
employeeRepository.save(employee);
```

Validation is triggered **automatically** if:

- Entity is managed
- Validation annotations are present

! If validation fails:

```
javax.validation.ConstraintViolationException
```

REST Exception Handling Best Practices

- ✓ Use `@RestControllerAdvice`
- ✓ Use custom exceptions
- ✓ Keep controllers thin
- ✓ Return meaningful HTTP status
- ✓ Return structured JSON
- ✓ Never expose stack traces